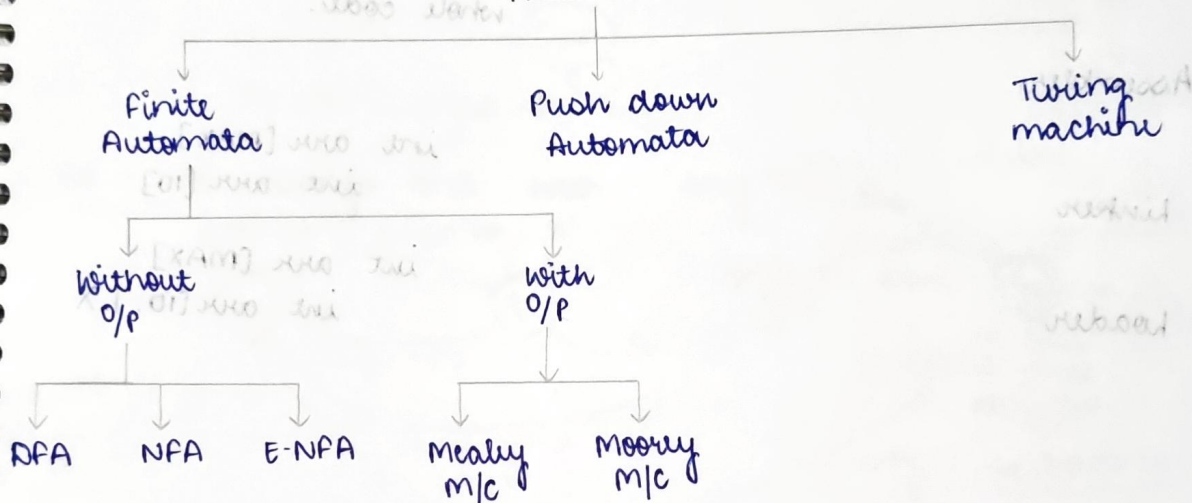


# Theory of Computation & Formal Languages (TCS - 402)

January 29, 2024

## Automata



## # Applications :-

→ Compilers

|    | 2 bits                    |   |
|----|---------------------------|---|
| +0 | $\frac{0}{1} \frac{0}{1}$ | 0 |
| +1 | $\frac{0}{1} \frac{1}{1}$ | 1 |
| -0 | $\frac{1}{1} \frac{0}{1}$ | 2 |
| -1 | $\frac{1}{1} \frac{1}{1}$ | 3 |

$$0 \text{ to } 3 = 0 \text{ to } 2^2 - 1$$

$$-1 \text{ to } +1 = -2^{2-1} - 1 \text{ to } 2^{2-1} - 1$$

eg: Turbo C, (earlier) data type  
int = 2B = 16 bits

unsigned = 0 to  $2^{16} - 1$

signed =  $-2^{15} - 1$  to  $2^{15} - 1$

## # usage of compiler :-

→ syntax check

→ if syntax is wrong, then it reports the error.

→ if all syntax is correct, then it converts the source code into assembly code.

→ assembler → object code  
(machine readable code)

Pre-processor

→ find & replace # directives. only prototypes  
# include <stdio.h>  
→ find stdio.h in bin, and replace this line with whole code.

Compiler

Assembler

Linker

Loader

loads the file in RAM

hard disk  
test.out/test.exe  
(links all files)

# define MAX 10  
int arr[MAX];  
int arr[10]  
# define MAX 10;  
int arr[MAX]  
int arr[10]; X

# Automata :-

start state

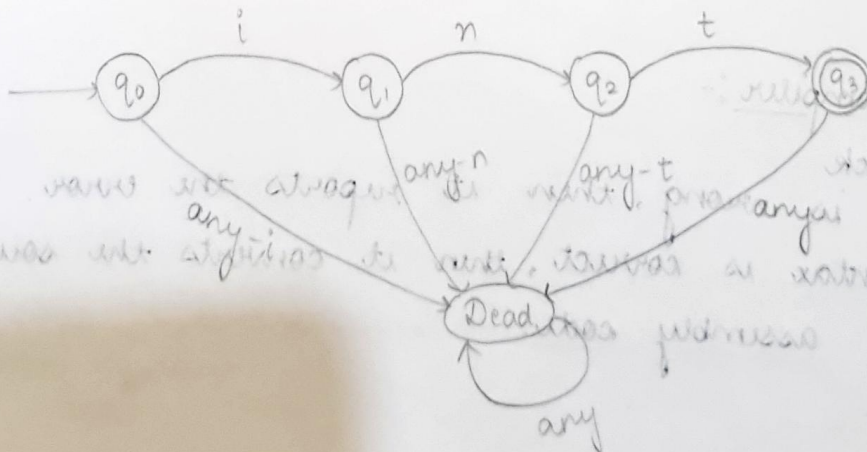
final state

loop  
(zero or  
n no. of  
times).

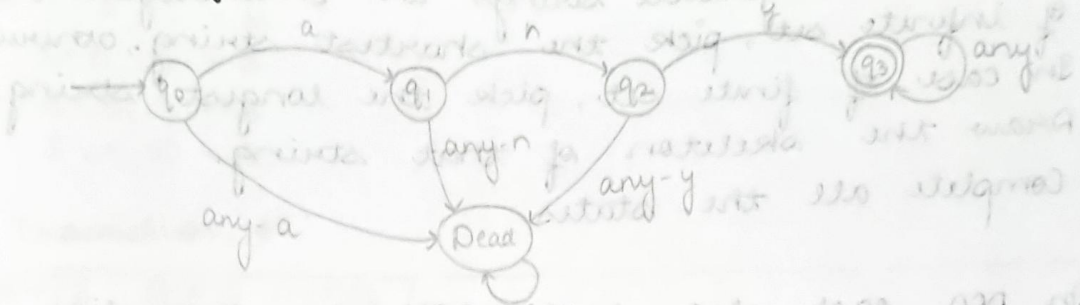
Transitions

\* Finite Automata (Deterministic):- DFA

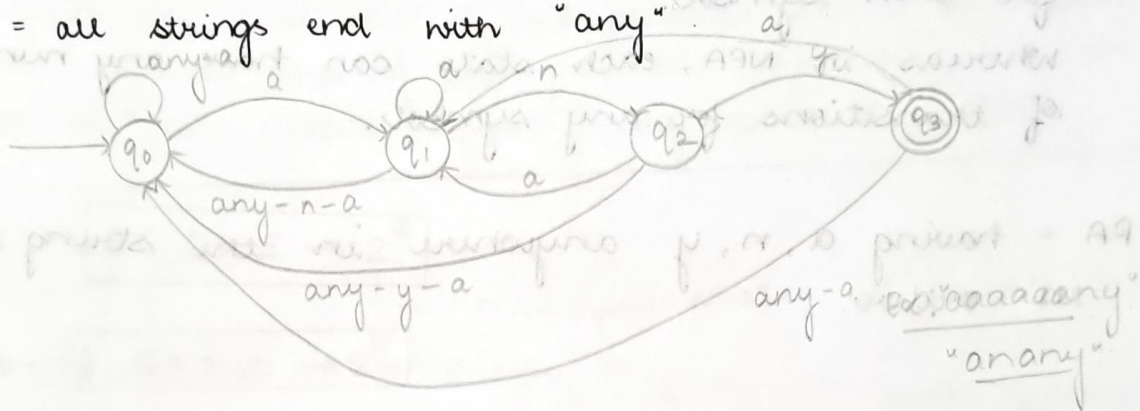
Q PA = only accepts "int" & rejects anything



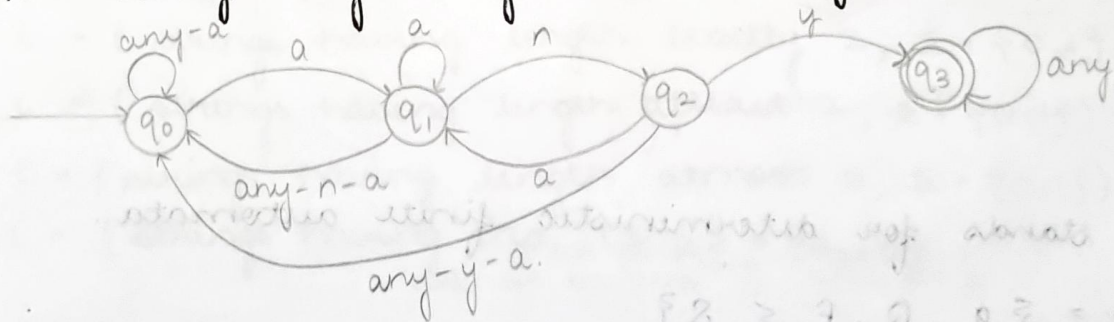
FA = string starts with "any"



FA = all strings end with "any"



FA = having "any" anywhere in string.



February 2, 24

→ symbols are represented by " $\Sigma$ ".

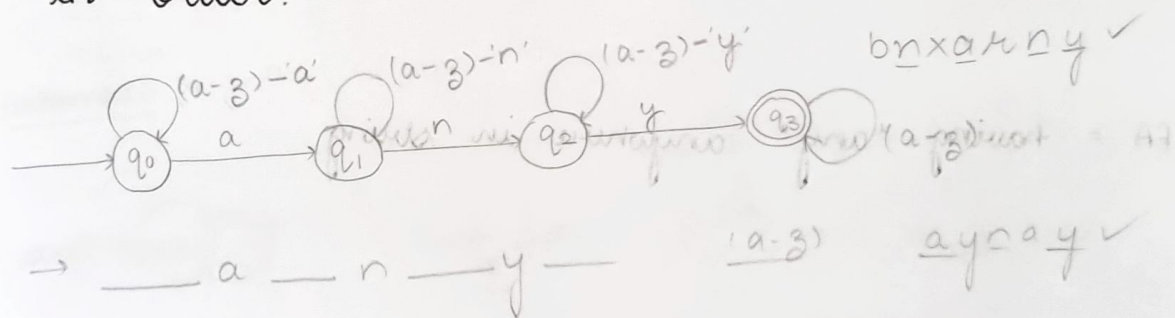
\* language → all the strings which are in our set.  
 → finite language  
 → infinite language.

\*  $\epsilon$  → special string of length 0.  
 (Epsilon)

- STEP 1: Write all the possible strings in chronological order.
- STEP 2: If infinite set, pick the shortest string, otherwise In case of finite set, pick the longest string.
- STEP 3: Draw the skeleton of that string.
- STEP 4: Complete all the states

NOTE: In DFA, each state has exactly one transition for each symbol. whereas in NFA, each state can have any number of transitions for any symbol.

Q PA = having a, n, y anywhere in the string but in order.



# DFA

DFA = stands for deterministic finite automata.

$$D = \{ q_0, Q, F, \Sigma, \delta \}$$

Annotations:

- $q_0$ : starting state
- $Q$ : finite set of all states
- $F$ : finite set of final state
- $\Sigma$ : finite set of allowed symbols
- $\delta$ : Transition function

\* In every machine, there is only 1 starting state.

\*  $F \subseteq Q$  and  $q_0 \in Q$ .

Example :-



$$Q = \{ A, B \}$$

$$F = \{ B \}$$

$$\Sigma = \{ 0, 1 \}$$

$\delta$  (current state, input symbol) = new state

$$\rightarrow \delta(A, 1) = A$$

$$\delta(A, 0) = B$$

$$\delta(B, 1) = B$$

$$\delta(B, 0) = A$$

Transition fn:

DFA  $\rightarrow \boxed{\delta: Q \times \Sigma \rightarrow Q}$

$\rightarrow$  from one state out of all possible state on reading one symbol out of all possible symbols, we can go to one state from any possible state

NFA  $\rightarrow \boxed{\delta: Q \times \Sigma \rightarrow 2^Q}$

$\rightarrow$  multiple states in power set

ENFA  $\rightarrow \boxed{\delta: Q \times \Sigma \cup E \rightarrow 2^Q}$

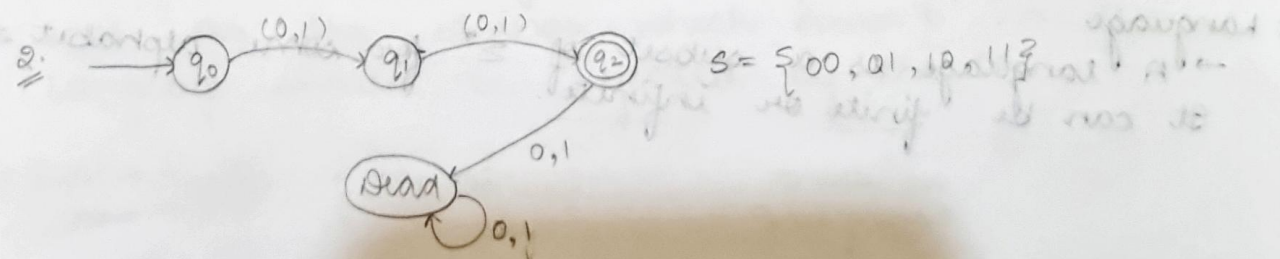
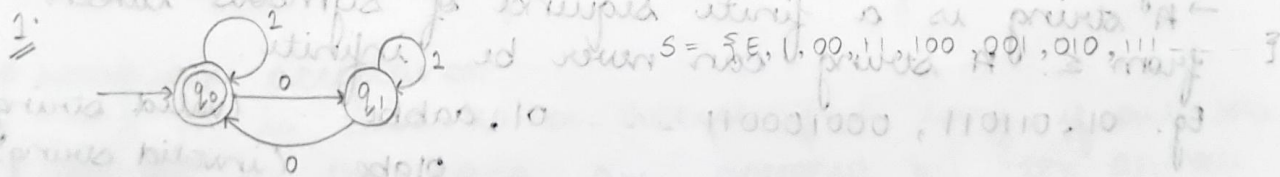
$\underline{Q} \ L = \{ \text{strings having even 0's, } \Sigma = \{0, 1\} \}$

$\underline{Q} \ L = \{ \text{strings having length exactly 2, } \Sigma = \{0, 1\} \}$

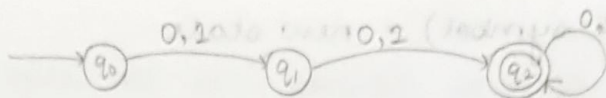
$\underline{Q} \ L = \{ \text{strings having length atleast 2, } \Sigma = \{0, 1\} \}$

$\underline{Q} \ L = \{ \text{strings having length atmost 2, } \Sigma = \{0, 1\} \}$

$\underline{Q} \ L = \{ \text{strings having odd 1's, } \Sigma = \{0, 1\} \}$

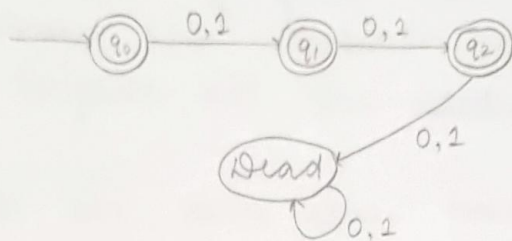


3.



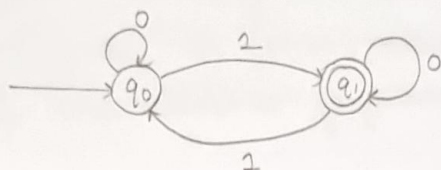
$$S = \{00, 01, 10, 11, 010, 100, \dots\}$$

4.



$$S = \{ \epsilon, 1, 0, 11, 00, 10, 01 \}$$

5.



$$S = \{1, 10, 01, 100, 001, 010, 111, \dots\}$$

February 06, 24

### \* Symbol

→ Basic building block of any machine, it can be any letter, any digits or numbers or even pictures.

### \* Alphabet

→ An alphabet is any finite set of symbols.

Eg.  $\Sigma = \{0, 1\}$

$\Sigma = \{a, b, \dots, z\}$  etc.

### \* String

→ A string is a finite sequence of symbols taken from  $\Sigma$ . A string can never be infinite.

Eg. 01, 011011, 000100011, ..., 01, aabbc, ... (valid string)  
01abc (invalid string)

### \* Language

→ A language is a subset of  $\Sigma^*$  for some alphabet  $\Sigma$ .  
It can be finite or infinite.

↓  
all strings  
having length  $\geq$

→ strings length  
at least 2.

for  $\Sigma = \{0, 1\}$

### \* Kleen

→ The Kleen star  $\Sigma^*$  is a unary operator on a set of symbols or strings,  $\Sigma$ , that gives the infinite set of all possible strings of all possible length over  $\Sigma$ .

$$\Sigma = \{0, 1\}$$

$$\Sigma^0 = \{\epsilon\}$$

$$\Sigma^1 = \{0, 1\}$$

$$\Sigma^2 = \{00, 01, 10, 11\}$$

$$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$$

$$\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, 011, \dots\}$$

### \* Kleen closure / plus

→ The set  $\Sigma^+$  is the infinite set of all possible strings of all possible length over  $\Sigma$ .

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

$$\Sigma^+ = \Sigma^* - \Sigma^0$$

### \* string acceptance

→ A string is said to be accepted by FA, if we are able to reach the final state starting from initial state upon scanning the entire string.

### \* language acceptance

→ A FA is said to be accepting a lang. if all the strings in the lang. are accepted by the FA as well as the strings which aren't in the language should be rejected by the FA.

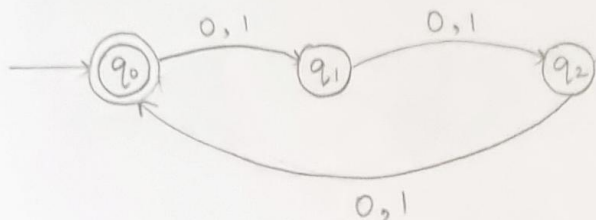
### \* $|w|$ → length of a string

Q  $|w| \% 3 \equiv 0$

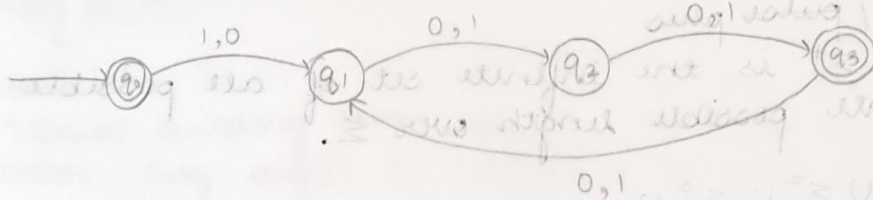
or  $|w| \bmod 3 \equiv 0$

or  $|w| \equiv 0 \bmod 3$

$L = \{ \epsilon, 001, 110, 100, 110011, \dots \}$



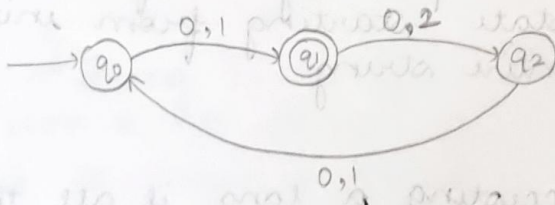
\* minimum states DFA



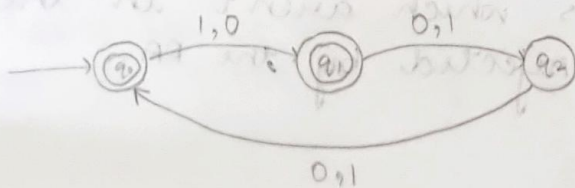
\* less optimized

Q  $|w| \% 3 \equiv 1$

1, 4, 7, 10, ...



Q  $|w| \% 3 \equiv 0 \text{ or } 1$

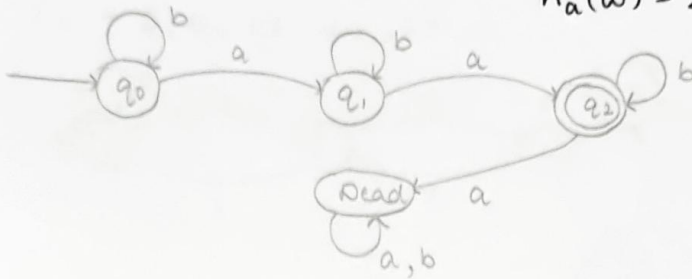


Q  $|w| \% 3 \equiv 0 \text{ or } 1$

↳ Not possible

Q Draw a minimal state DFA for all strings having nos of 'a' = 2,  $\Sigma = \{a, b\}$ . or

$$n_a(w) = 2.$$



$S = \{aa, aba, aab, \dots\}$

$b^* a b^* a b^*$

Q  $n_a(w) \geq 2$

Q  $n_a(w) \leq 2$

Q mDFA where  $w \in \{a, b\}^*$  ( $\Sigma = \{a, b\}$ )

$$n_a(w) \bmod 2 = 0$$

vs

$$n_b(w) \bmod 2 = 0$$

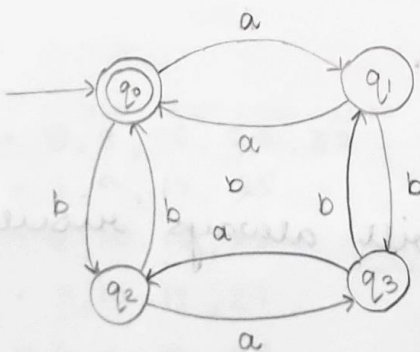
Q  $n_a(w) \bmod 3 = 0$  vs  $n_b(w) \bmod 2 = 0$

Q DFA  $\Sigma = \{0, 1\}$ , if  $w$  when interpreted as decimal number must be divisible by 3.

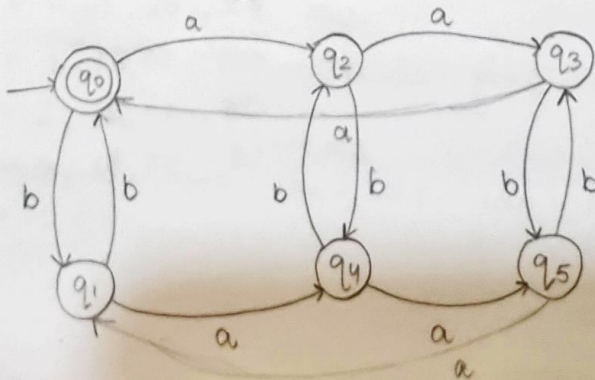
eg.  $(101)_2 = (5)_{10}$  X

$(110)_2 = (6)_{10}$  ✓

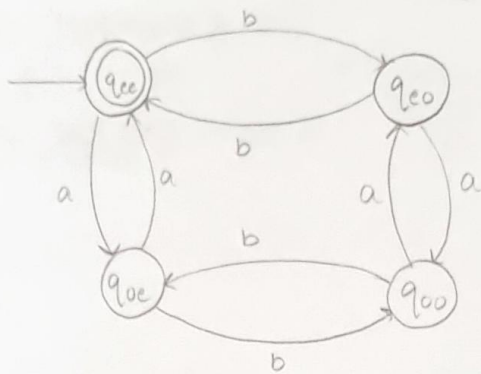
1.



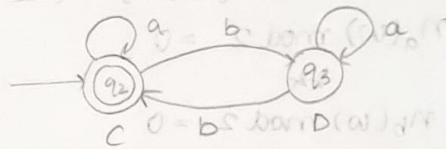
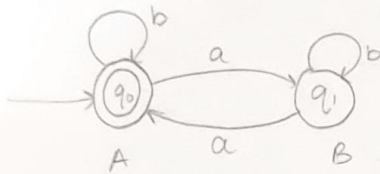
2.



February 09, 24



using cross product :-



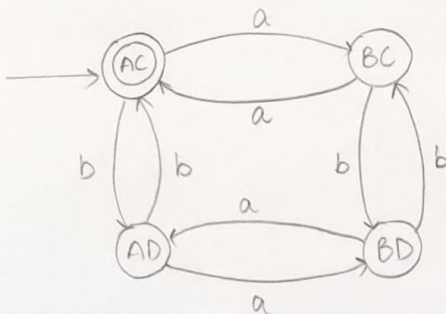
$$n_a(w) \bmod 2 = 0$$

$$n_b(w) \bmod 2 = 0$$

$$S_1 = \{A, B\}$$

$$S_2 = \{C, D\}$$

$$S_1 \times S_2 = \{(A, C), (A, D), (B, C), (B, D)\}$$



\* Cross product of two DFA will always result in a DFA.

\*  $11 \rightarrow 3$   $110 \rightarrow 6$   
 - when we put zero after any no., it will double

$11 \rightarrow 3$   $101 \rightarrow 7$   
 - when we put one after any no., it will double and added by 1.

$$C_0 = 3, 6, 9, 12, 15 \dots$$

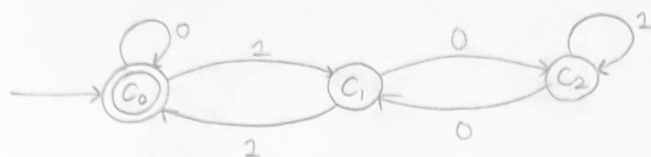
$$C_1 = 4, 7, 10, 13, 16 \dots$$

$$C_2 = 5, 8, 11, 14, 17 \dots$$

$$3 \times 2 = 6$$

$$2 \times 2 + 1 = 5 \rightarrow C_1$$

$$4 \times 2 = 8 \rightarrow C_2$$



Q strings interpreted as decimal are completely divisible by 5.

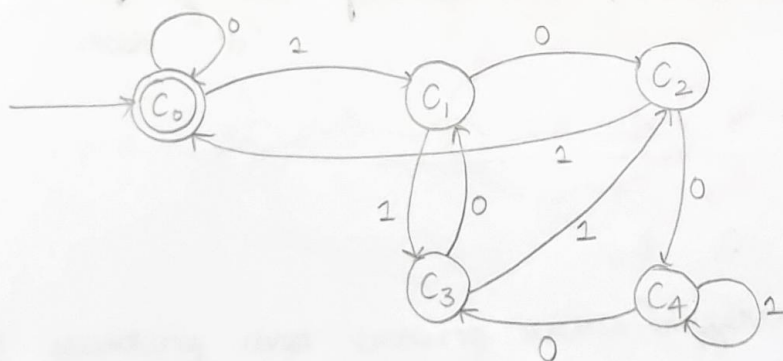
$$C_0 = 0, 5, 10, 15, 20, 25 \dots$$

$$C_1 = 1, 6, 11, 16, 21, 26 \dots$$

$$C_2 = 2, 7, 12, 17, 22, 27 \dots$$

$$C_3 = 3, 8, 13, 18, 23, 28 \dots$$

$$C_4 = 4, 9, 14, 19, 24, 29 \dots$$



$$C_0 = 0, 8, 16, 24, 32 \dots 40$$

$$C_1 = 1, 9, 17, 25$$

$$C_2 = 2, 10, 18, 26$$

$$C_3 = 3, 11, 19, 27$$

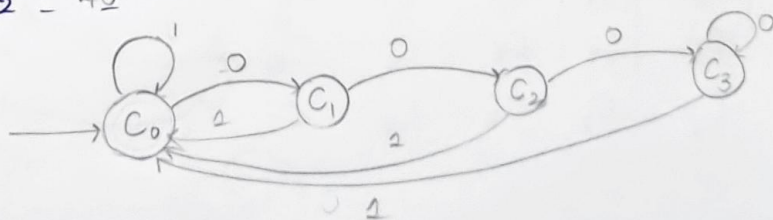
$$C_4 = 4, 12, 20, 28$$

$$C_5 = 5, 13, 21, 29$$

$$C_6 = 6, 14, 22, 30$$

$$C_7 = 7, 15, 23, 31$$

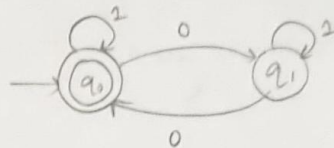
8 \*



Assumption - string should have atleast length 3.

- State diagram
- Tuples
- Tabulation method.

|         | 0     | 1     |
|---------|-------|-------|
| → $q_0$ | $q_1$ | $q_0$ |
| $q_1$   | $q_0$ | $q_1$ |



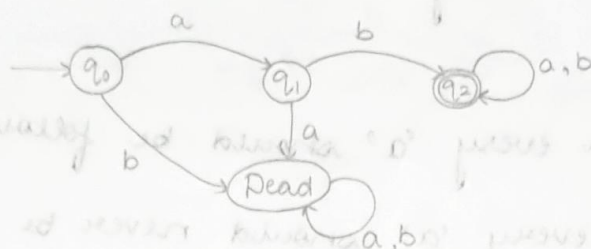
Ques.  $(n)_5 \% 7 = 0 \Rightarrow (n)_{10}$ .

String given in radix 5, divisible by 7 gives remainder 0.

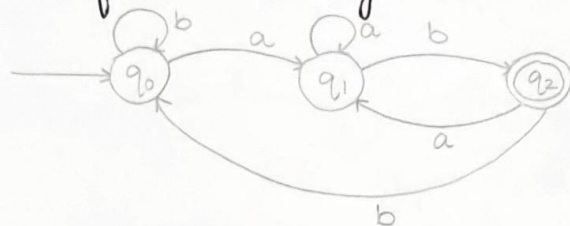
|         | 0     | 1     | 2     | 3     | 4     |
|---------|-------|-------|-------|-------|-------|
| → $C_0$ | $C_0$ | $C_1$ | $C_2$ | $C_3$ | $C_4$ |
| $C_1$   | $C_5$ | $C_6$ | $C_0$ | $C_1$ | $C_2$ |
| $C_2$   | $C_3$ | $C_4$ | $C_5$ | $C_6$ | $C_0$ |
| $C_3$   | $C_1$ | $C_2$ | $C_3$ | $C_4$ | $C_5$ |
| $C_4$   | $C_6$ | $C_0$ | $C_1$ | $C_2$ | $C_3$ |
| $C_5$   | $C_4$ | $C_5$ | $C_6$ | $C_0$ | $C_1$ |
| $C_6$   | $C_2$ | $C_3$ | $C_4$ | $C_5$ | $C_6$ |

February 18, 21

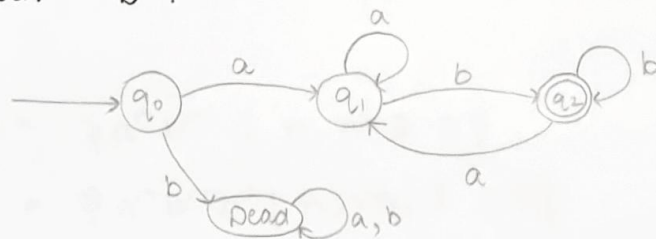
Q Draw a minimized DFA for all strings starting with 'ab'.  
 $\Sigma = \{a, b\}$



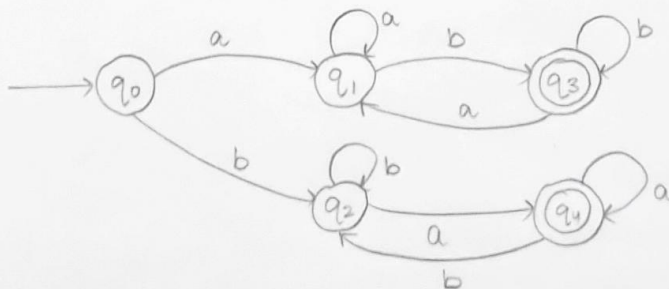
Q Set of all strings that end with 'ab'.



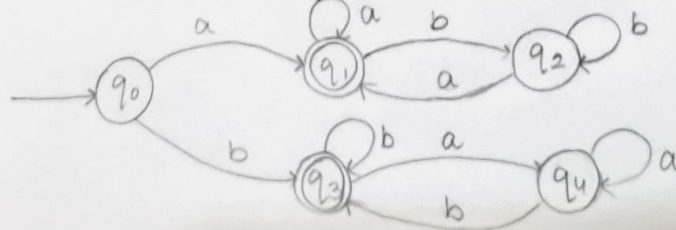
Q Set of all strings that starts with 'a' and ends with 'b'.



Q starting and ending with different symbols.  $\Sigma = \{a, b\}$   
 $S = \{ab, ba, a-b, b-a, \dots\}$



Q starting & ending with same symbol.



1.  $L$  = strings in which every 'a' is followed by 'b'.

$$S = \{ \epsilon, ab, abbb \dots ab, b, abab \dots \}$$

2.  $L$  = strings in which every 'a' should never be followed by b.

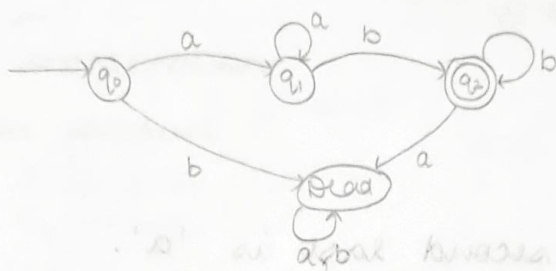
$$S = \{ a, ba, \dots bba \dots, bbb \dots, \epsilon \}$$

3.  $L$  = strings in which every 'a' should be followed by 'bb'.

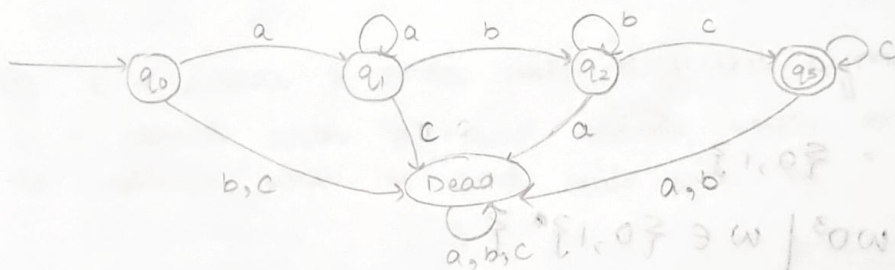
4.  $L$  = strings in which every 'a' should never be followed by 'bb'.

Q  $\Sigma = \{a, b\}$

$L = \{a^n b^m \mid n, m \geq 1\}$



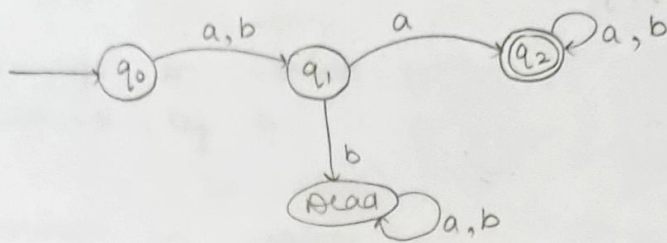
Q  $L = \{a^n b^m c^l \mid n, m, l \geq 1\}$



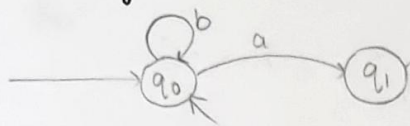
1.  $L = \{a^n b^m \mid n, m \geq 0\}$

2.  $L = \{a^n b^m c^l \mid n, m, l \geq 0\}$

Q.  $L =$  strings in which second symbol from LHS is 'a'



Q.  $L =$  strings in which second last is 'a'.



Q. DFA  $\Sigma = \{0, 1\}$

$$L = \{0^3 \mid w0^3 \mid w \in \{0, 1\}^*\}$$

$$S = 0001 \underline{\quad} 000$$

$\uparrow$   
 $\Sigma^*$

\* Introduction.

ASSIGNMENT-1  
 Name (TCS-402)  
 Uni Roll No.  
 Sec  
 Class Roll No

February 16, 24

## # Properties of DFA.

- i) Cross product
- ii) Union

iv) Complement.

v) Reverse.

iii) Concatenation.

- Cross product  $\leftarrow$  closed

$D_1 \rightarrow$  DFA

$D_2 \rightarrow$  DFA

$D_1 \times D_2 \rightarrow D \rightarrow$  guaranteed DFA

- Union.

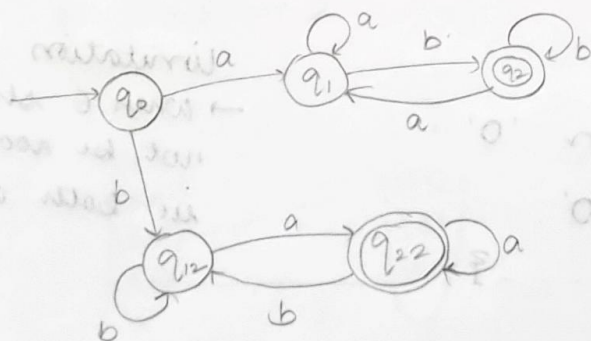
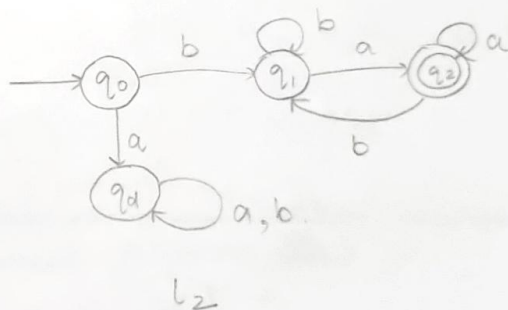
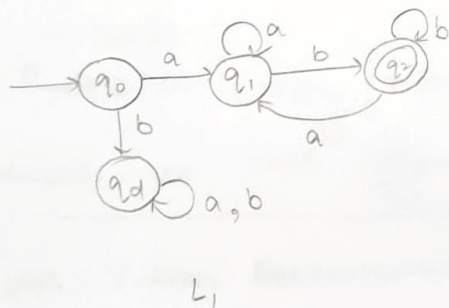
$D_1 \cup D_2 = D$

DFA    DFA    DFA

Eg.  $L =$  starts & ends with different symbols.

$L_1 =$  starts with 'a' and ~~starts~~<sup>ends</sup> with 'b'.

$L_2 =$  starts with 'b' and ends with 'b'.



\* maximum states in after union :-

$\rightarrow$  remains same

$\rightarrow$  decrease by 1 or more.

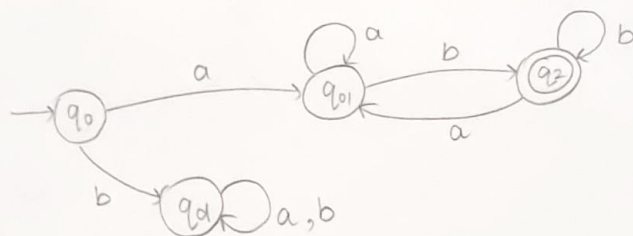
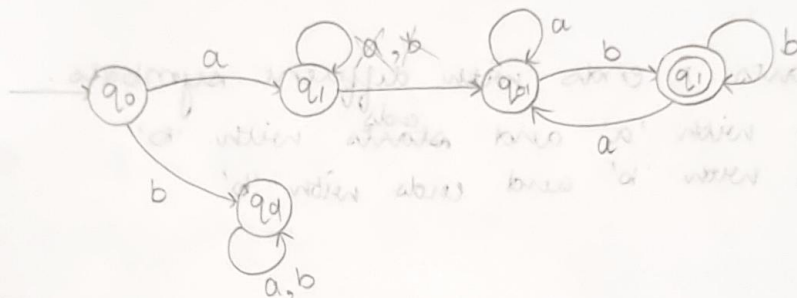
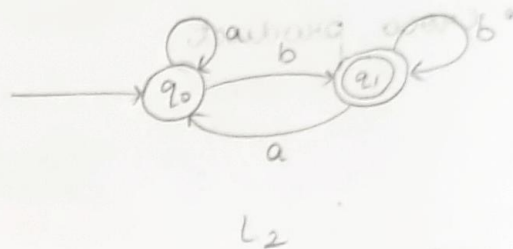
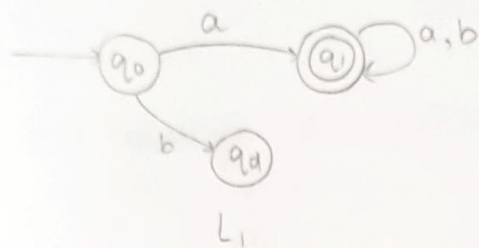
$\rightarrow$  increase by 1.

## • Concatenation

$L = \{^+ \text{starts with 'a' and ends with 'b'}\}$

$L_1 = \text{starts with 'a'}$

$L_2 = \text{ends with 'b'}$



## • Complement

$L = \text{doesn't contain '0'}$

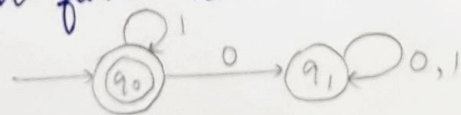
$L_1 = \text{does contain '0'}$

$L_1 = \{0, 00, 01, 10, \dots\}$

\* 2A



\* All final state has to be made non-final and all non-final state has to be final state.



Limitation  
→ When  $\epsilon$  should not be accepted in both concatenation

NOT

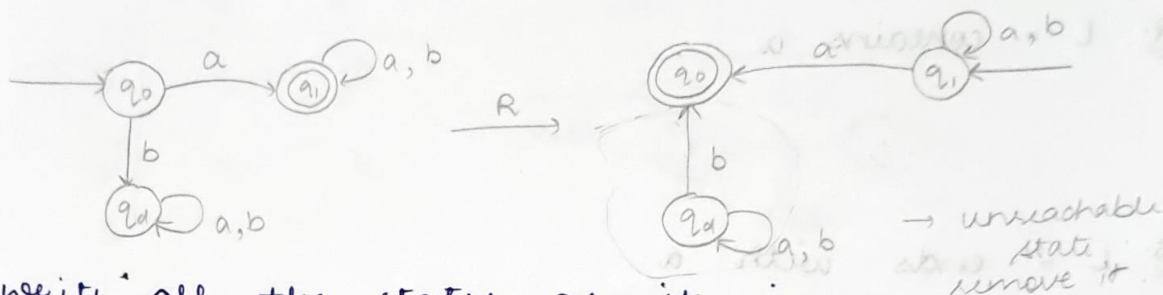
## • Reversal

$$a^*wb \xrightarrow{R} b^*w^*a$$

$L_1$  = starts with 'a'

$L_1 = \{a, aa, ab, abba, \dots\}$

$L_1^R = \{a, aa, ba, bba, \dots\}$



\* i) write all the states as it is.

ii) Make final state  $\rightarrow$  initial state & initial state  $\rightarrow$  final state.

iii) Reverse the direction of all transitions.

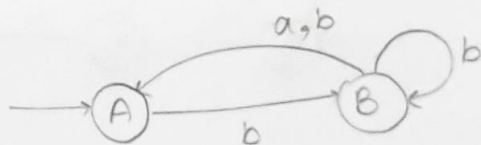
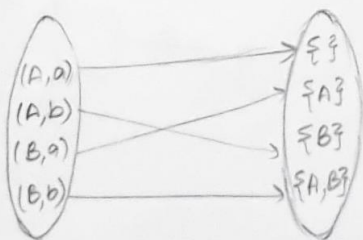
\*  $L_1 \rightarrow$  DFA

$L_1^R \nrightarrow$  DFA (may / may not)

\* Reversal is not closed.

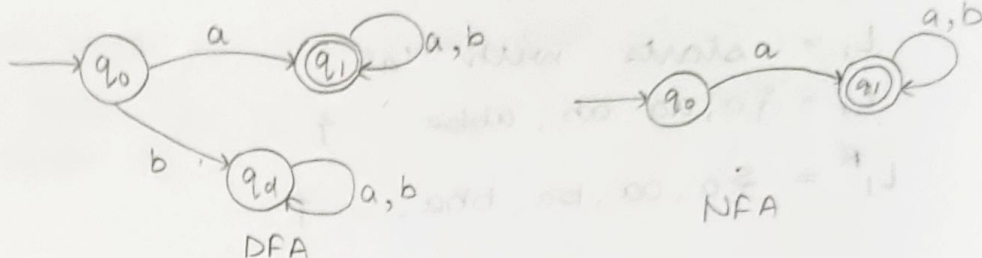
## # NFA (Non-Deterministic Finite Automata)

$$\delta: Q \times \Sigma \rightarrow 2^Q$$

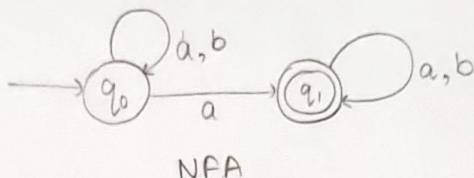


NOTE :- Every DFA is a NFA but the vice-versa isn't true.

1.  $L = \text{starts with 'a'}$



2.  $L = \text{contains a}$



3.  $L = \text{ends with a}$

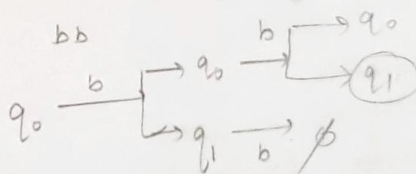


\* In NFA, there is parallel programming.

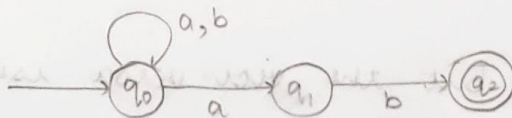
4.  $L = \text{starts with 'ab'}$



5.  $L = \text{contains ab}$



6.  $L = \text{ends with 'ab'}$



7.  $L = 0^n 1^n \mid n \geq 1$

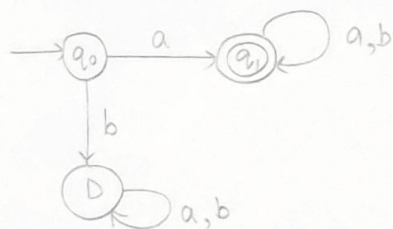
→ Not Possible.

# # Conversion of NFA to DFA

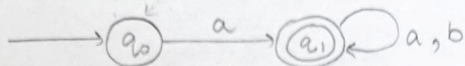


|                   | a            | b      |
|-------------------|--------------|--------|
| $\rightarrow q_0$ | $(q_1, q_0)$ | $\phi$ |
| $q_1^*$           | $q_1$        | $q_1$  |

|                   | a           | b           |
|-------------------|-------------|-------------|
| $\rightarrow q_0$ | $q_1$       | $\emptyset$ |
| $q_1$             | $q_1$       | $q_1$       |
| $\emptyset$       | $\emptyset$ | $\emptyset$ |



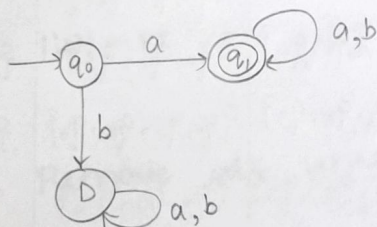
# # Conversion of NFA to DFA



|                  | a                                  | b              |
|------------------|------------------------------------|----------------|
| → q <sub>0</sub> | (q <sub>1</sub> , q <sub>0</sub> ) | ∅              |
| q <sub>1</sub> * | q <sub>1</sub>                     | q <sub>1</sub> |

|                  | a              | b              |
|------------------|----------------|----------------|
| → q <sub>0</sub> | q <sub>1</sub> | D              |
| q <sub>1</sub>   | q <sub>1</sub> | q <sub>1</sub> |
| D                | D              | D              |

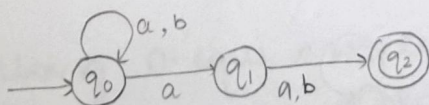
1. Copy the initial state.



February 20, 24

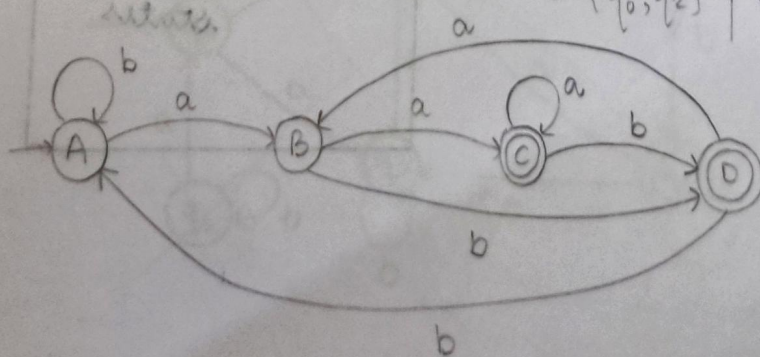
Q second last symbol is a.  $\Sigma = \{a, b\}$

NFA -

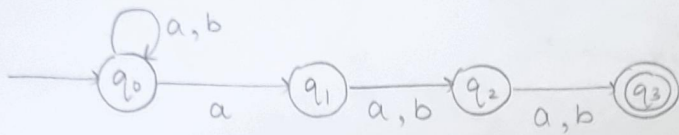


|                  | a                                  | b                 |
|------------------|------------------------------------|-------------------|
| → q <sub>0</sub> | {q <sub>0</sub> , q <sub>1</sub> } | {q <sub>0</sub> } |
| q <sub>1</sub>   | {q <sub>2</sub> }                  | {q <sub>2</sub> } |
| *q <sub>2</sub>  | ∅                                  | ∅                 |

|                  | a                                                                | b                                               |
|------------------|------------------------------------------------------------------|-------------------------------------------------|
| → q <sub>0</sub> | {q <sub>0</sub> , q <sub>1</sub> } <sub>B</sub>                  | {q <sub>0</sub> } <sub>A</sub>                  |
| q <sub>1</sub>   | {q <sub>0</sub> , q <sub>1</sub> } <sub>C</sub>                  | {q <sub>0</sub> , q <sub>2</sub> } <sub>D</sub> |
| q <sub>2</sub>   | {q <sub>0</sub> , q <sub>1</sub> , q <sub>2</sub> } <sub>C</sub> | {q <sub>0</sub> , q <sub>2</sub> } <sub>D</sub> |
| D                | {q <sub>0</sub> , q <sub>2</sub> } <sub>B</sub>                  | {q <sub>0</sub> } <sub>A</sub>                  |



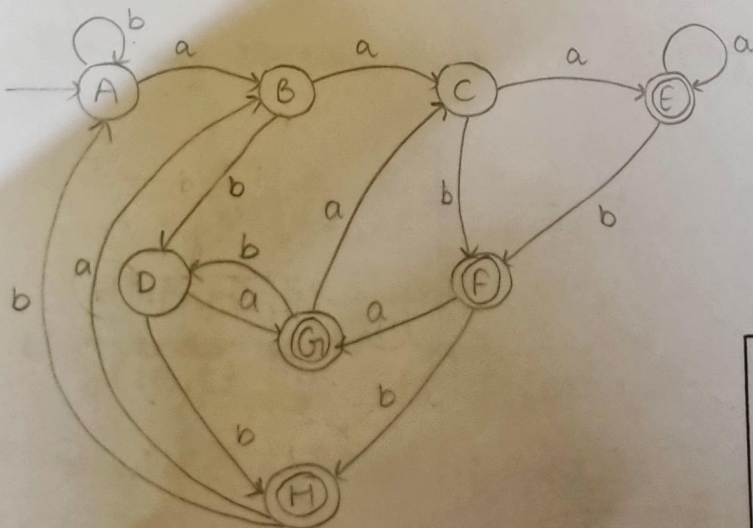
Ques Draw a DFA for  $\Sigma = \{a, b\}$  where  
 $L = \{\text{all strings having 3rd last symbol as 'a'}\}$ .



|                   | a              | b           |
|-------------------|----------------|-------------|
| $\rightarrow q_0$ | $\{q_0, q_1\}$ | $\{q_0\}$   |
| $q_1$             | $\{q_2\}$      | $\{q_2\}$   |
| $q_2$             | $\{q_3\}$      | $\{q_3\}$   |
| $*q_3$            | $\emptyset$    | $\emptyset$ |

|                            | a                        | b                   |
|----------------------------|--------------------------|---------------------|
| A $\rightarrow q_0$        | $\{q_0, q_1\}$           | $\{q_0\}$           |
| B $\{q_0, q_1\}$           | $\{q_0, q_1, q_2\}$      | $\{q_0, q_2\}$      |
| C $\{q_0, q_1, q_2\}$      | $\{q_0, q_1, q_2, q_3\}$ | $\{q_0, q_2, q_3\}$ |
| D $\{q_0, q_2\}$           | $\{q_0, q_1, q_3\}$      | $\{q_0, q_3\}$      |
| E $\{q_0, q_1, q_2, q_3\}$ | $\{q_0, q_1, q_2, q_3\}$ | $\{q_0, q_2, q_3\}$ |
| F $\{q_0, q_2, q_3\}$      | $\{q_0, q_1, q_3\}$      | $\{q_1, q_3\}$      |
| G $\{q_0, q_1, q_3\}$      | $\{q_0, q_1, q_2\}$      | $\{q_0, q_2\}$      |
| H $\{q_0, q_3\}$           | $\{q_0, q_1\}$           | $\{q_1\}$           |

$\{d, n\} \geq \dots$



nFA  $\rightarrow n$  states  
 $\downarrow$   
 DFA  $\rightarrow \max 2^n$  states.

## # Minimization of DFA (Algorithm)

(p, q) states can be merged if,

$$\delta(p, w) \in F \quad \& \quad \delta(q, w) \in F$$

$$\delta(p, w) \notin F \quad \& \quad \delta(q, w) \notin F$$

→ if  $|w| = 0$  → 0-equivalence  
 $|w| = 1$  → 1-equivalence  
 $|w| = 2$  → 2-equivalence  
⋮  
 $|w| = n$  → n-equivalence.

Step 1: Remove all unreachable states.

Step 2: Draw the transition table of the DFA.

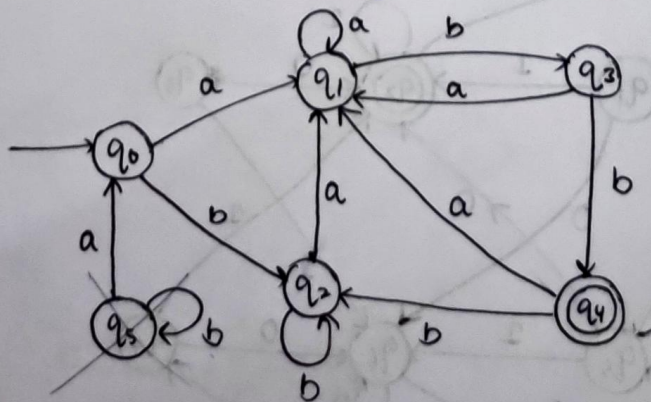
Step 3: Write 0-equivalence

↳ separate group for non-final states & final states

Step 4: using 0-equi. write 1 eq.  
using 1 eq. write 2 eq.

using  $(n-1)^{th}$  eq, write n-eq.

till 2 adjacent equivalence are same.



|                   | a     | b       |                 | a   | b    |
|-------------------|-------|---------|-----------------|-----|------|
| $\rightarrow q_0$ | $q_1$ | $q_2$   | $\rightarrow A$ | $B$ | $A'$ |
| $q_1$             | $q_1$ | $q_3$   | $B$             | $B$ | $C$  |
| $q_2$             | $q_1$ | $q_2$   | $\rightarrow$   |     |      |
| $q_3$             | $q_1$ | $q_4^*$ | $C$             | $B$ | $D$  |
| $*q_4$            | $q_1$ | $q_2$   | $*D$            | $B$ | $A'$ |

0-equivalence:  $[q_0, q_1, q_2, q_3]$   $[q_4]$

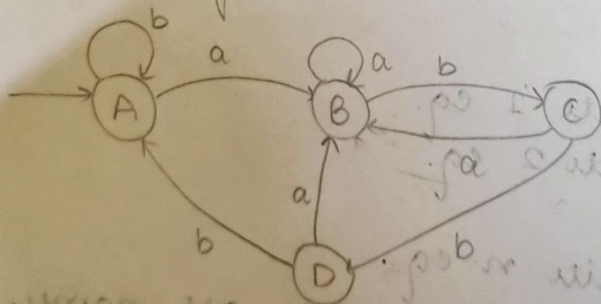
$\rightarrow$  Groups can never be merged, only divide.

1-equivalence:  $[q_0, q_1, q_2]$   $[q_3]$   $[q_4]$

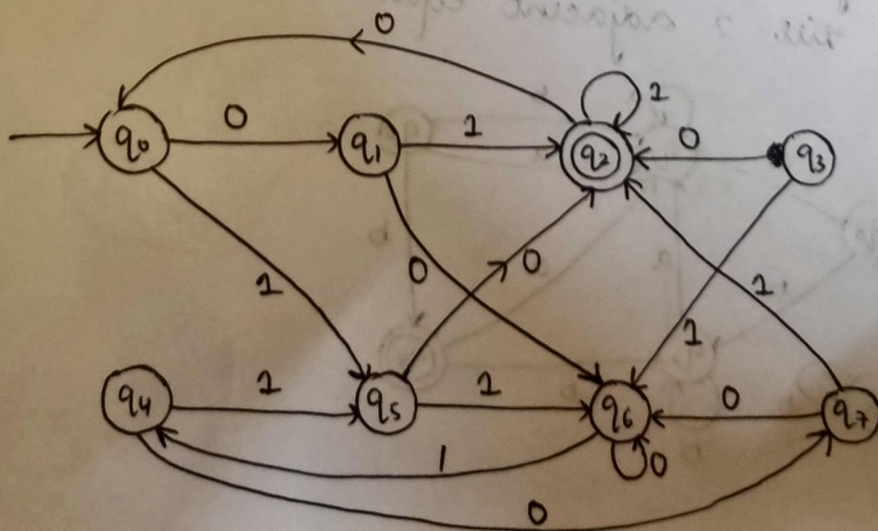
2-equivalence:  $[q_0, q_2]$   $[q_1]$   $[q_3]$   $[q_4]$

3-equivalence:  $[q_0, q_2]$   $[q_1]$   $[q_3]$   $[q_4]$

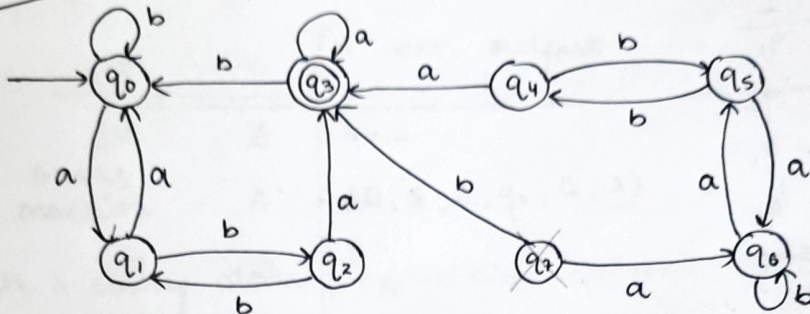
$\rightarrow$  check vertically



Ques: 1.



Ques 2.

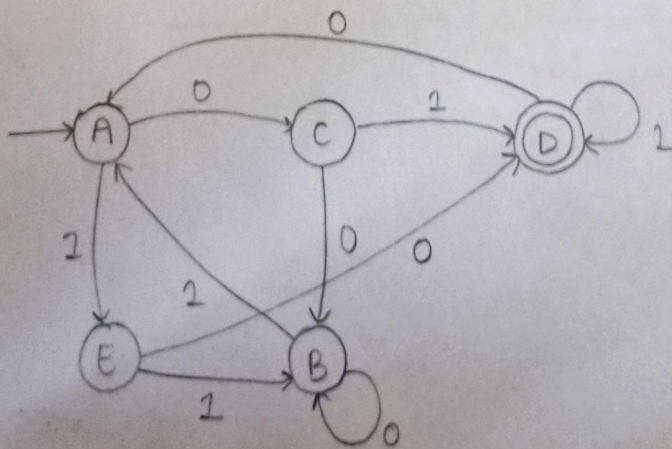


1.

|                   | 0       | 1       |
|-------------------|---------|---------|
| $\rightarrow q_0$ | $q_1$   | $q_5$   |
| $q_1$             | $q_6$   | $q_2^*$ |
| $* q_2$           | $q_0$   | $q_2^*$ |
| $q_4$             | $q_7$   | $q_5$   |
| $q_5$             | $q_2^*$ | $q_6$   |
| $q_6$             | $q_6$   | $q_4$   |
| $q_7$             | $q_6$   | $q_2^*$ |

|                 | 0 | 1 |
|-----------------|---|---|
| $\rightarrow A$ | C | E |
| B               | B | A |
| C               | B | D |
| $* D$           | A | D |
| E               | D | B |

- 0 equivalence:  $[q_0, q_1, q_4, q_5, q_6, q_7]$   $[q_2]$
- 1 equivalence:  $[q_0, q_4, q_5]$   $[q_1, q_7]$   $[q_2]$   $[q_6]$
- 2 equivalence:  $[q_0, q_4]$   $[q_5]$   $[q_1, q_7]$   $[q_2]$   $[q_6]$
- 3 equivalence:  $[q_0, q_4]$   $[q_5]$   $[q_1, q_7]$   $[q_2]$   $[q_6]$

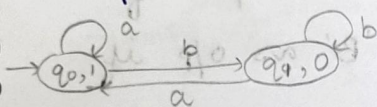


February 27, 24

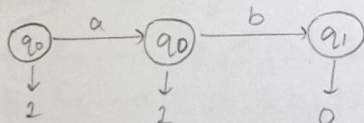
# FA with output

## moore machine

→ each & every state will produce one output.



I: ab



O: 110

→ Input length = n  
Output length = n+1

→  $\lambda: Q \rightarrow \Delta$

$(Q, \Sigma, \delta, q_0, \Delta, \lambda)$

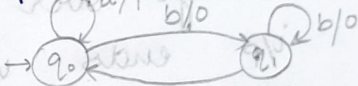
$\Delta \rightarrow$  o/p alphabet

$\lambda \rightarrow$  o/p function

$\delta: Q \times \Sigma \rightarrow Q$

## mealy machine

each & every transition will produce output.



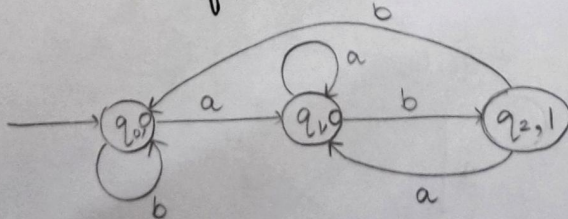
I: ab

O: 10

Input length = n  
Output length = n.

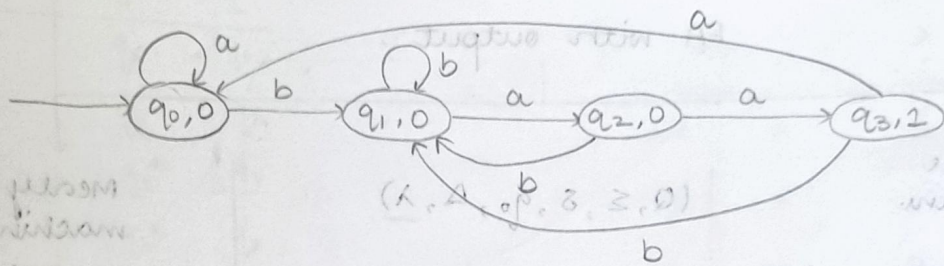
$\lambda: Q \times \Sigma \rightarrow \Delta$

Ques construct a moory m/c that takes set of all strings over  $\{a, b\}$  as i/p & prints '1' as o/p for every occurrence of 'ab' as a substring.

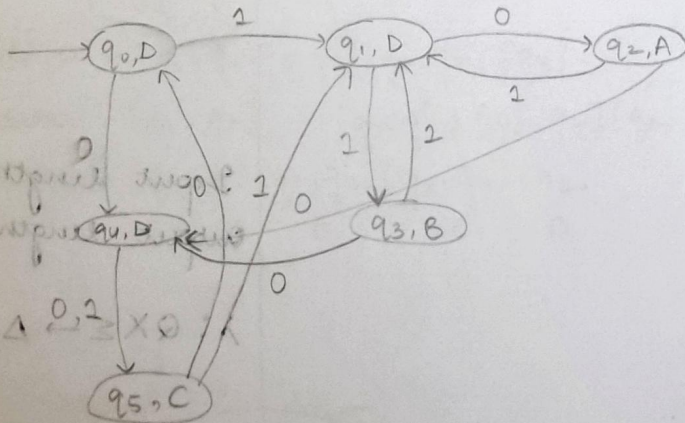


Ques. moory m/c for "baa".

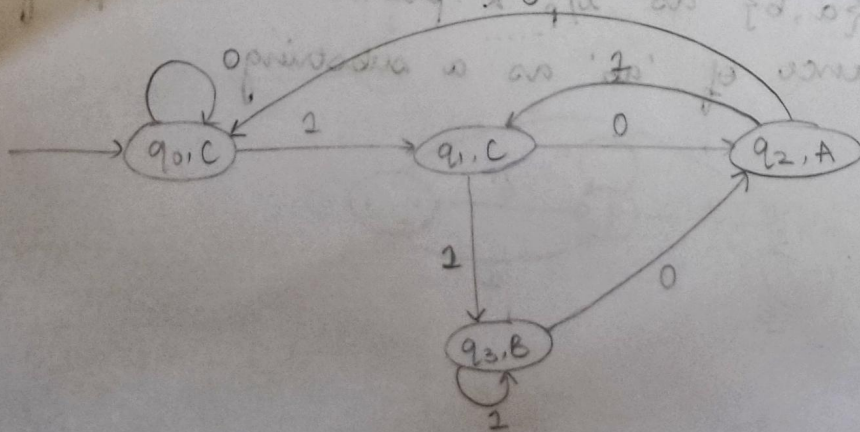
FS, FS



Ques. Construct a moory m/c that takes set of all strings over  $\{0,1\}$  & produces 'A' as o/p if i/p ends with '10' or produces 'B' as o/p if i/p ends with '11', otherwise 'C'.



Ques. Construct a moory m/c that takes set of all strings over  $\{0,1\}$  & produces 'A' as o/p if i/p ends with '10' or produces 'B' as o/p if i/p ends with '11', otherwise 'C'.

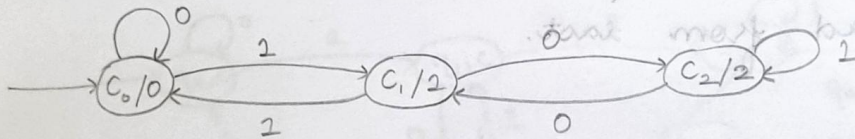


Ques. Moony machine for all  $\{0,1\}$  which when interpreted as decimal no. produces modulo 3 as output.

$C_0 \rightarrow 0, 3, 6, 9, 12, 15, \dots$  110

$C_1 \rightarrow 1, 4, 7, 10, 13, 16, \dots$

$C_2 \rightarrow 2, 5, 8, 11, 14, 17, \dots$



Ques. Base 4 . O/P = 1.5.

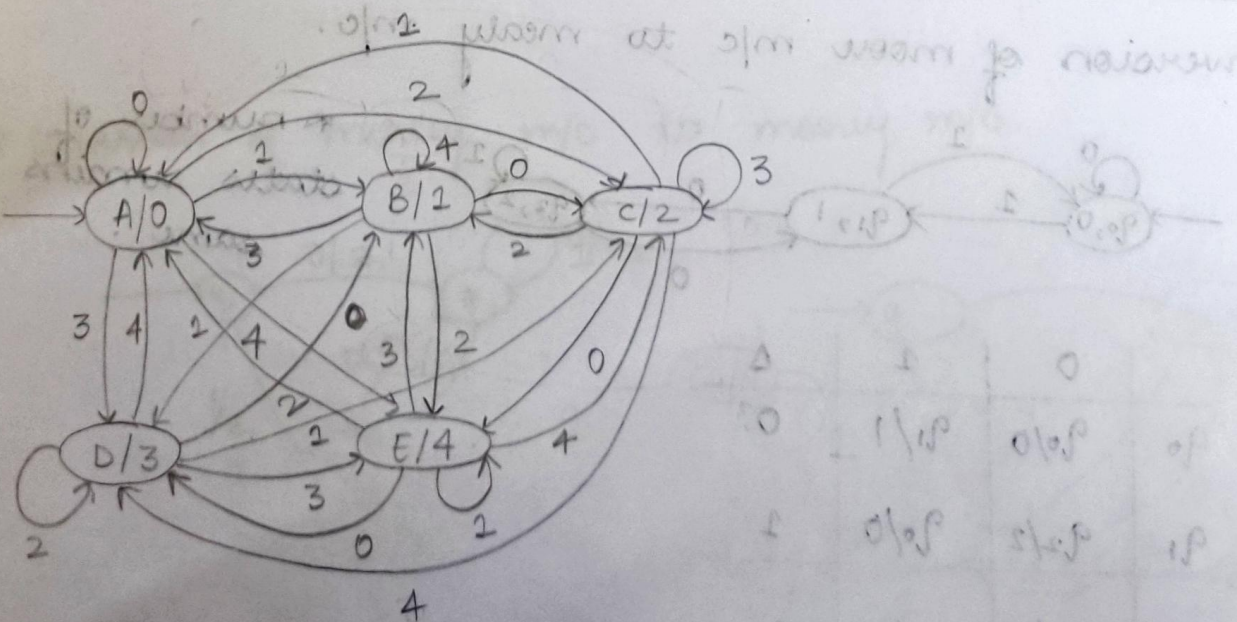
A  $C_0 \rightarrow 0, 5, 10, 15, 20, 25, \dots$

B  $C_1 \rightarrow 1, 6, 11, 16, 21, 26, \dots$

C  $C_2 \rightarrow 2, 7, 12, 17, 22, 27, \dots$

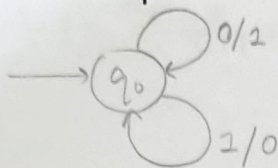
D  $C_3 \rightarrow 3, 8, 13, 18, 23, 28, \dots$

E  $C_4 \rightarrow 4, 9, 14, 19, 24, 29, \dots$

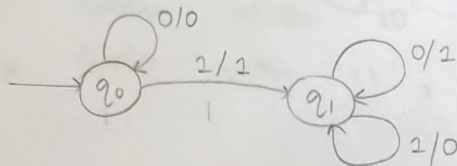


# # Mealy Machine.

Q for the complement of I/P.



Q O/P should be 2's complement.  
Input is read from last.



For 2's complement :-

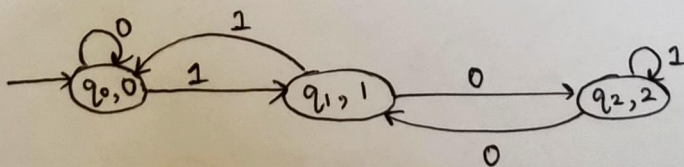
→ 11011000

o/p → 00101000

[From last let zero's be zero, when we get first 1, it remains 1, after that every symbol is complemented.]

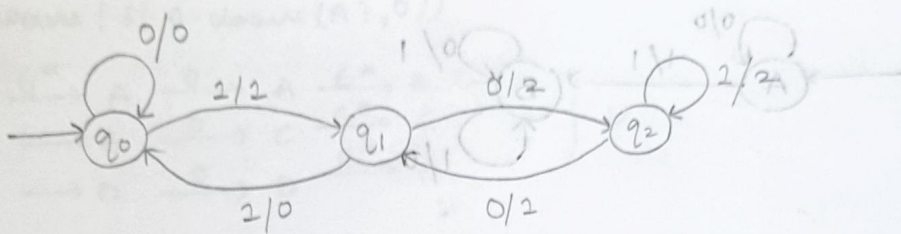
March 01, 24 :

## # Conversion of moore m/c to mealy m/c.

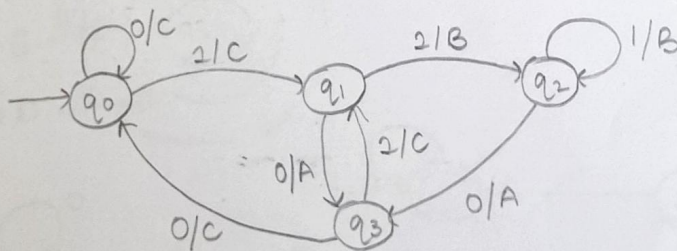
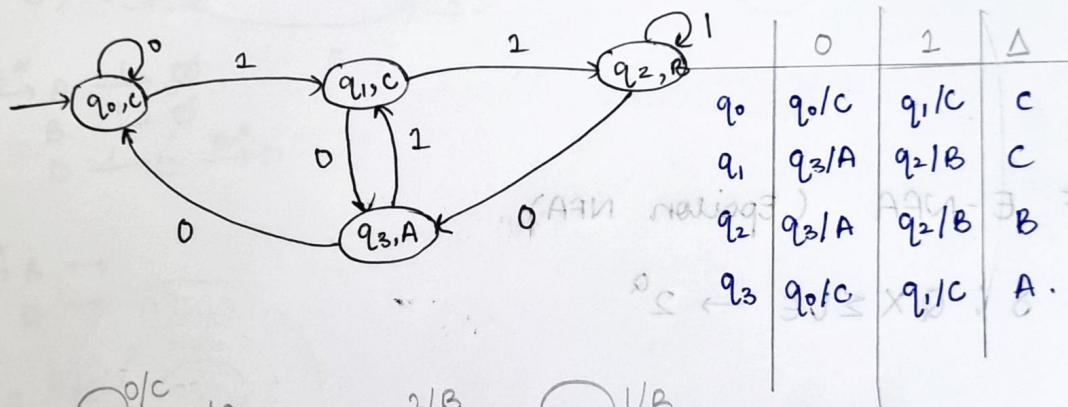


\* number of states remains same.

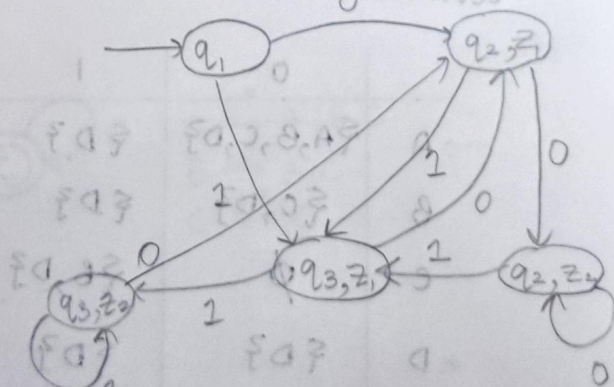
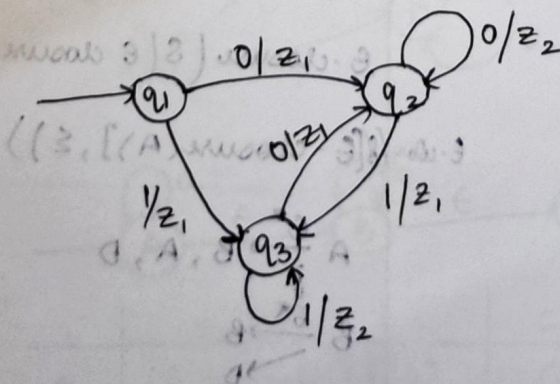
|                | 0                 | 1                 | Δ |
|----------------|-------------------|-------------------|---|
| q <sub>0</sub> | q <sub>0</sub> /0 | q <sub>1</sub> /1 | 0 |
| q <sub>1</sub> | q <sub>2</sub> /2 | q <sub>0</sub> /0 | 1 |
| q <sub>2</sub> | q <sub>1</sub> /1 | q <sub>2</sub> /2 | 2 |



Question:-  $10 \rightarrow A$  ,  $11 \rightarrow B$  , otherwise  $\rightarrow C$ .



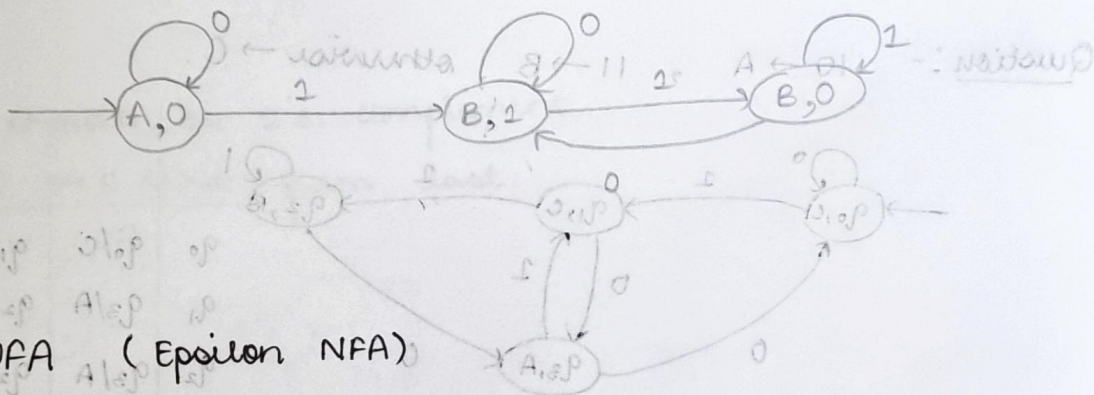
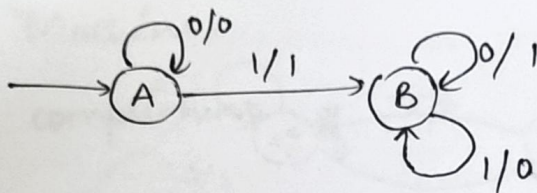
# Conversion of many m/c to many m/c.



\* Best case no. of states are same.

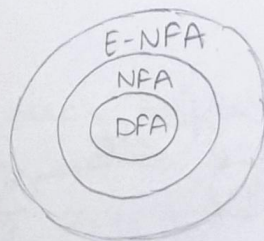
\* No. of increment state  $\geq$  no's of input  $\times$  no. of states.

Q

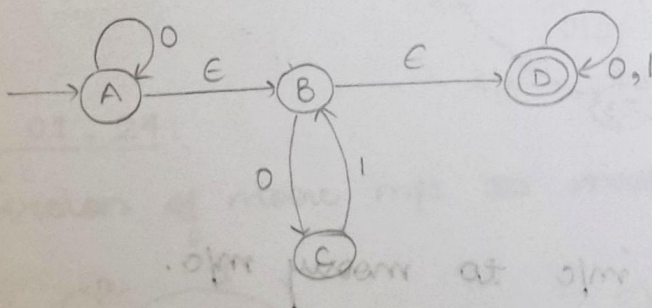


# E-NFA (Epsilon NFA)

$$\delta : Q \times \Sigma \cup \epsilon \rightarrow 2^Q$$



Eg:-



Convert this to NFA.

|     | 0            | 1      |
|-----|--------------|--------|
| → A | {A, B, C, D} | {D}    |
| B   | {C, D}       | {D}    |
| C   | ∅            | {B, D} |
| * D | {D}          | {D}    |

\* E-NFA → NFA → DFA.

$$\epsilon\text{-closure}(\{ \epsilon\text{-closure}(Q) \})$$

$$\epsilon\text{-closure}(\{ \epsilon\text{-closure}(A) \}, \epsilon)$$

$$A \xrightarrow{\epsilon^*} B, A, D$$

$$B \xrightarrow{\epsilon^*} B$$

$$\quad \quad \quad \rightarrow D$$

$$C \xrightarrow{\epsilon^*} C$$

$$D \xrightarrow{\epsilon^*} D$$

$\epsilon$ -closure ( $\delta(\epsilon$ -closure( $A$ ),  $0$ ))

$$\left. \begin{array}{l} A \xrightarrow{\epsilon^*} A \xrightarrow{0} A \xrightarrow{\epsilon^*} A, B, D \\ \quad \rightarrow B \xrightarrow{0} C \xrightarrow{\epsilon^*} C \\ \quad \rightarrow D \xrightarrow{0} D \xrightarrow{\epsilon^*} D \end{array} \right\} U$$

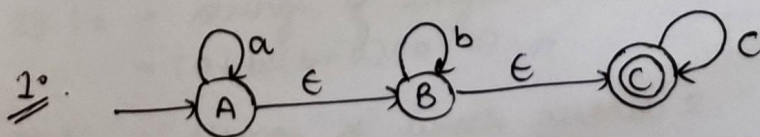
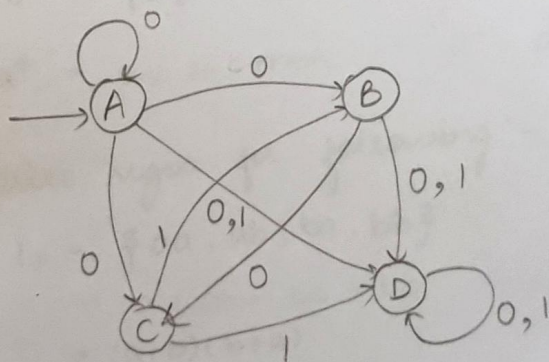
~~$$\begin{array}{l} A \xrightarrow{\epsilon^*} A \xrightarrow{1} A \xrightarrow{\epsilon^*} A, B, D \\ \quad \rightarrow B \xrightarrow{1} B \xrightarrow{\epsilon^*} B, C \\ \quad \rightarrow D \xrightarrow{1} D \xrightarrow{\epsilon^*} D, C \end{array}$$~~

~~$$\begin{array}{l} A \xrightarrow{\epsilon^*} A \xrightarrow{1} \emptyset \\ \quad \rightarrow B \xrightarrow{1} \emptyset \\ \quad \rightarrow D \xrightarrow{1} D \xrightarrow{\epsilon^*} D \end{array}$$~~

~~$$\begin{array}{l} B \xrightarrow{\epsilon^*} B \rightarrow \\ \quad D \rightarrow \end{array}$$~~

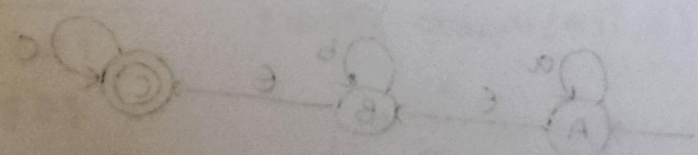
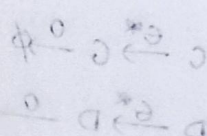
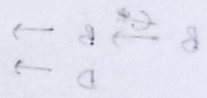
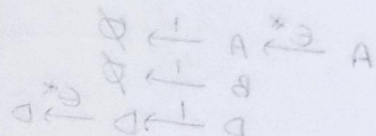
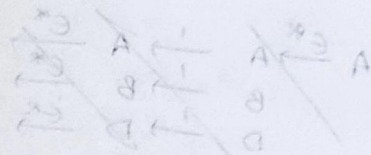
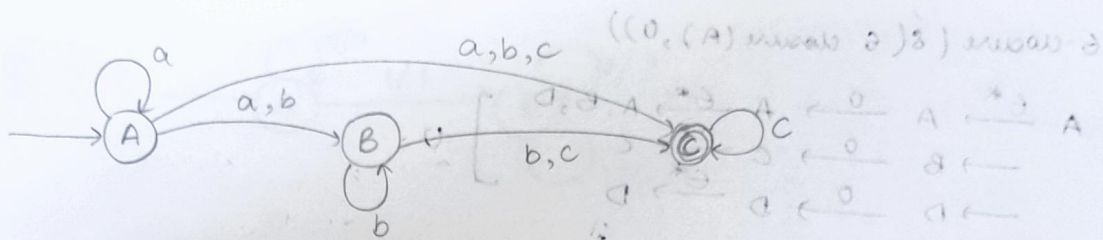
~~$$C \xrightarrow{\epsilon^*} C \xrightarrow{0} \emptyset$$~~

~~$$D \xrightarrow{\epsilon^*} D \xrightarrow{0} D$$~~



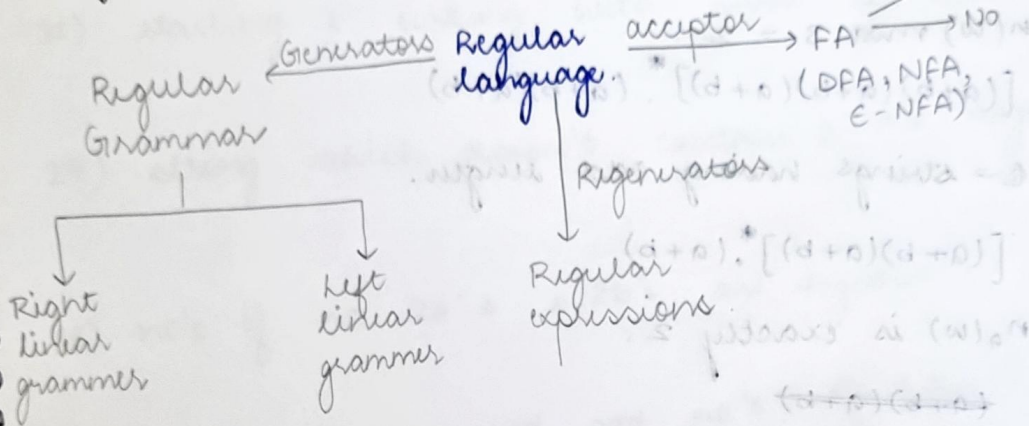
$$\begin{array}{l} A \xrightarrow{\epsilon^*} A, B, C \\ B \xrightarrow{\epsilon^*} B, C \\ C \xrightarrow{\epsilon^*} C \end{array}$$

|   | a           | b           | c |
|---|-------------|-------------|---|
| A | (A, B, C)   | (B, C)      | C |
| B | $\emptyset$ | (B, C)      | C |
| C | $\emptyset$ | $\emptyset$ | C |



March 05, 24

### # Regular Expressions:-



operators:-

- + (union) (or)
- (concatenate)
- \* (Kleen closure)

→  $\phi = \{\}$  → Nothing to accept → No final state  
 $\epsilon = \{\epsilon\}$  → Something to accept → final state  
 $a^+ = \text{one or more}$  →  $a^+ = a^* \cdot \epsilon$

Q write regex for following:-

2)  $L_1 = \{aa, ab, ba, bb\}$   
 $\rightarrow aa + ab + ba + bb$   
 $= (a+b)(b+a)$

2)  $l_2$  = strings of length 3.  
 $= (a+b)(b+a)(a+b)$

3)  $L_3$  = strings of length atleast 2.  
 $= (a+b)(a+b)(a+b) \dots$

4)  $L_4$  = strings of length at most 2.  
 $= (e+a+b)(e+a+b)$

5)  $L_5$  = strings having even length.  
 $= [(a+b)(a+b)]^*$

NOTE :- avoid  $a^+$   
write  $a \cdot a^*$

6)  $L_6 =$  strings having length which is divisible by 3.  
 $= [(a+b)(a+b)(a+b)]^*$

7)  $L_7 = n(w) \bmod 3 = 2$ .  
 $= [(a+b)(a+b)(a+b)]^* \cdot (a+b)(a+b)$

8)  $L_8 =$  strings having odd length.  
 $= [(a+b)(a+b)]^* \cdot (a+b)$

9)  $L_9 = n_a(w)$  is exactly 2.  
 $= \cancel{(a+b)(a+b)}$   
 $= b^* a b^* a b^*$

10)  $L_{10} =$  strings in which a's are atleast 2.  
 $= \cancel{(b^* a b^*)} + \cancel{(a^* b^*)}$   
 $= \cancel{b^* a b^* a b^* (a^* b^*)^*}$   
 $= b^* a b^* a (b+a)^*$

11)  $L_{11} =$  a's are atmost 2.  
 $= b^* (\epsilon + a) b^* (\epsilon + a) b^*$

12)  $L_{12} =$  even no. of a's.  
 $= (b^* a b^* a b^*)^* + b^*$   
 $= b^* (b^* a b^* a b^*)^*$

13)  $L_{13} =$  starts with a.  
 $= a[a^* b^*]^*$

14)  $L_{14} =$  ends with a.

15) containing a.

- 25) starting & ending with diff symbol.
- 26) starting & ending with same symbol.
- 27) string which doesn't contain 2 a's together.
- 28) no's of no 2a's & 2b's are together.
- 29) even no. of 0's and odd no's of 1's.

### # Identities :-

$$\phi + R = R + \phi = R$$

$$\phi \cdot R = R \cdot \phi = \phi$$

$$\epsilon \cdot R = R \cdot \epsilon = R$$

$$\epsilon^* = \epsilon$$

$$\phi^* = \phi$$

$$\epsilon + R = R + \epsilon = R \cup \epsilon$$

$$\epsilon + RR^* = R^*R + \epsilon = R^*$$

$$(a+b)^* = (a^* + b^*)^* = (a^*b^*)^* = (a+b^*)^* = a^*(a^*b^*)^* = b^*(ab^*)^*$$

### # Regex to finite automata :-

$$\rightarrow \phi \rightarrow \text{single state}$$

$$\epsilon \rightarrow \text{single state with self-loop}$$

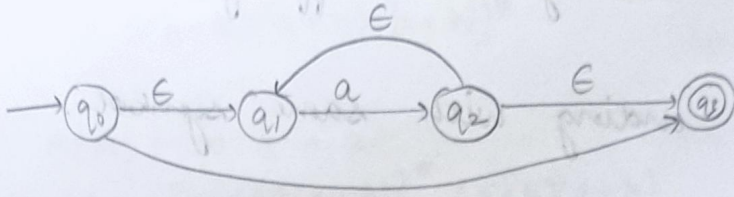
$$a \rightarrow \text{two states } q_0 \xrightarrow{a} q_1$$

$$a+b \rightarrow \text{two states } q_0 \xrightarrow{a,b} q_1$$

$$a \cdot b \rightarrow \text{three states } q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2$$

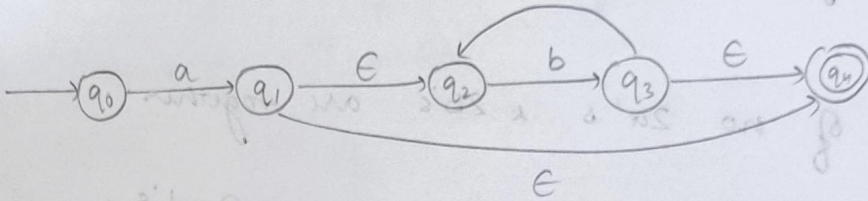
$$a^* \rightarrow \text{single state with self-loop labeled } a$$

$a^*$

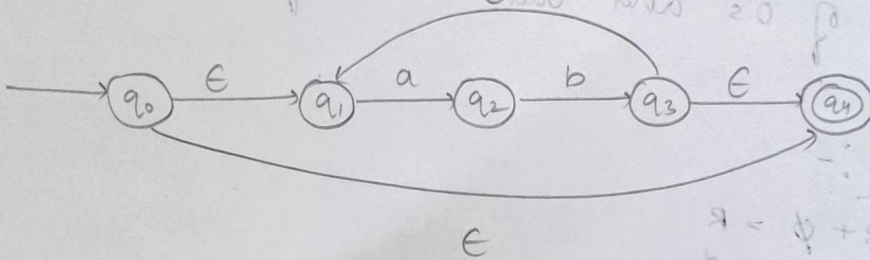


$ab^*$

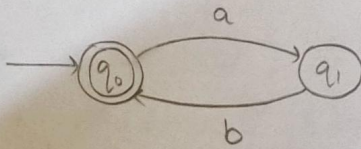
→



$(ab)^*$



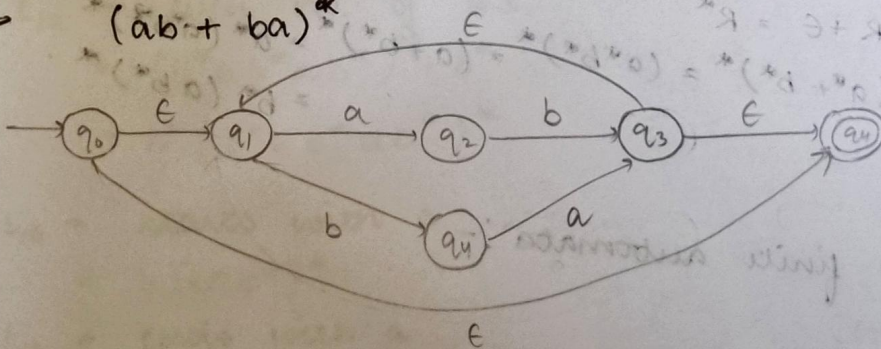
or



$$\begin{aligned} \phi + R &= R + \phi = R \\ \phi \cdot R &= R \cdot \phi = R \\ \epsilon \cdot R &= R \cdot \epsilon = R \\ \epsilon &= \phi \\ \phi &= \phi \end{aligned}$$

Ques:

$(ab + ba)^*$



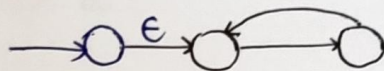
March 12, 24

# FA to RE

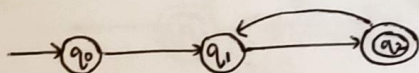


→

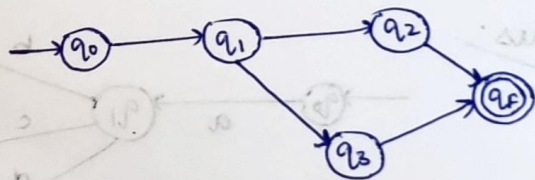
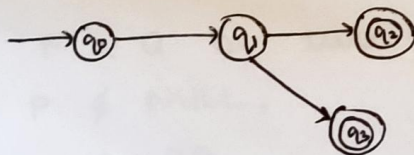
Step 1:- The start state should only have outgoing edges, if there is any incoming edge we have to make new start state through  $\epsilon$  transition.



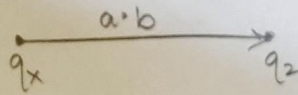
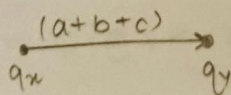
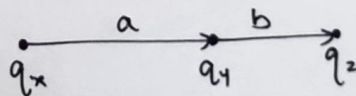
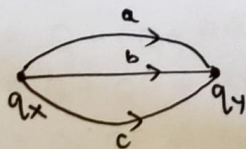
Step 2:- The final state shouldn't have any outgoing edges, if so introduce new final state using  $\epsilon$  transition.



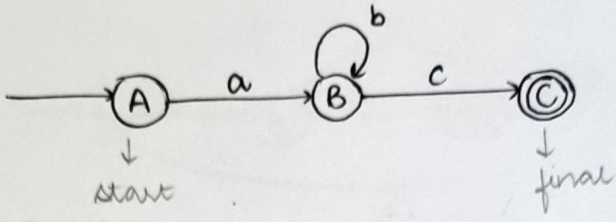
If there are more than one final state, join them all through  $\epsilon$  transition to one single final state.



Step 3:- Eliminate all the states between initial and final state, one by one.



Ques:

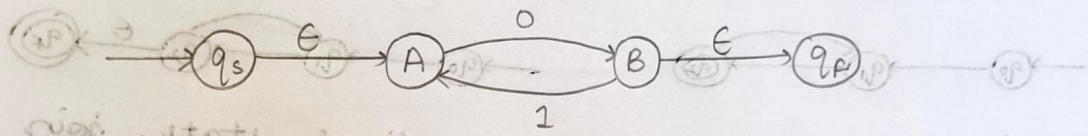
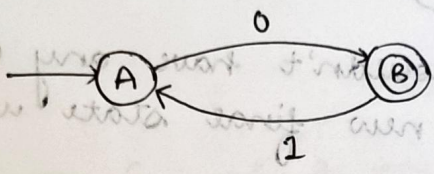


FR at A9 #

properties must have all states. states of  $w$  - 1 get  
 must on up to previous state if not in state  
 transition  $A \xrightarrow{a \cdot b^* \cdot c} C$  Ans:  $a \cdot b^* \cdot c$   
 $= ab^*c$

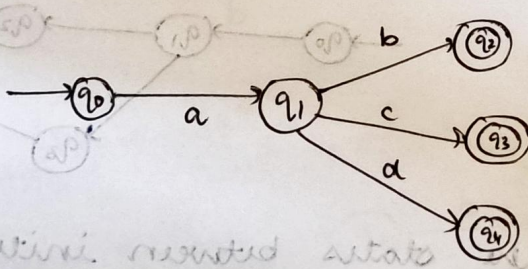


Ques:

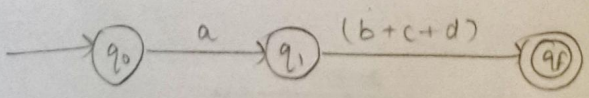
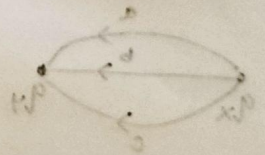
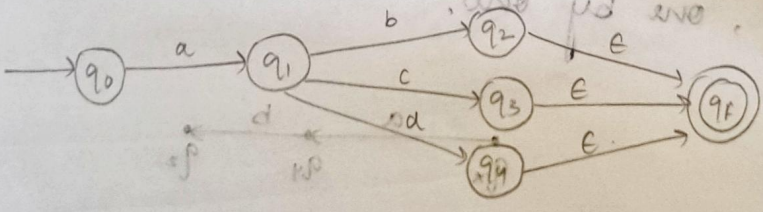


if states are not same  $\epsilon \cdot (01)^* \cdot 0 \cdot \epsilon$   
 up to at transition  $\epsilon$  Ans:  $(01)^* \cdot 0$  /  $0(10)^*$   
 states same

Ques:

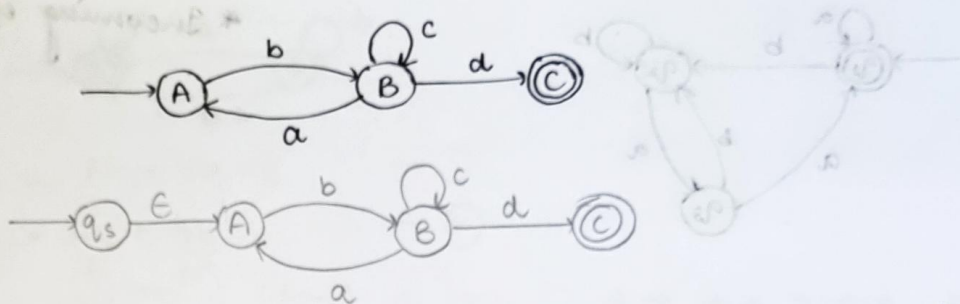


Eliminate all states between initial and final state. one of the states same



Ans:  $a \cdot (b+c+d)$

Ques:

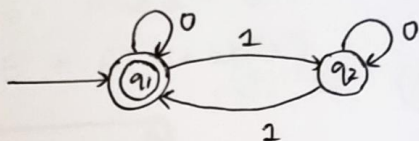


Ans: -  ~~$b(ab)^*c^*d$~~   
 $b(ab+c)^*d$

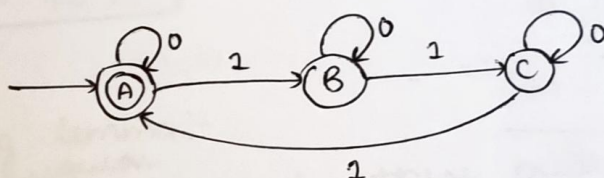
~~$(bc^*a)^*bc^*d$~~

~~$b(ab)^*c^*d$~~

Ques: 9



Ques 10:



# Aiden's Theorem

→ FA to RE

→ let  $P \times Q$  be two R.E.

if  $P \neq \text{NULL}$ ,

$$R = Q + RP$$

$$\Rightarrow \boxed{R = QP^*}$$

$$R = Q + RP$$

$$= Q + (Q + RP)P$$

$$= Q + QP + RP^2$$

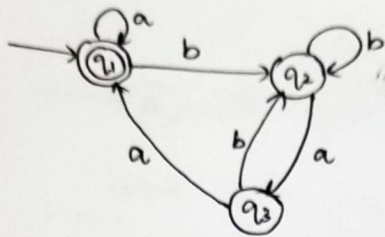
$$= Q + QP + (Q + RP)P^2$$

$$= Q + QP + QP^2 + QP^3 + \dots$$

$$R = Q[e + P + P^2 + P^3 + \dots]$$

$$R = QP^*$$

Ques:



\* Incoming edges.

$$q_1 = q_1 a + q_3 a + \epsilon \text{ (initial is final)} \quad \text{--- (1)}$$

$$q_2 = q_1 b + q_2 b + q_3 b$$

$$q_3 = q_2 a$$

$$\begin{aligned} & \text{--- (2)} \\ & \text{--- (3)} \end{aligned}$$

Put value of  $q_3$  from (3) into (2),

$$q_2 = q_1 b + q_2 b + q_2 a b$$

$$\frac{q_2}{R} = \frac{q_1 b}{Q} + \frac{q_2}{R} \left( \frac{b + ab}{P} \right)$$

so,

$$q_2 = q_1 b (b + ab)^*$$

Put (4) in (3),

$$q_3 = q_1 b (b + ab)^* a$$

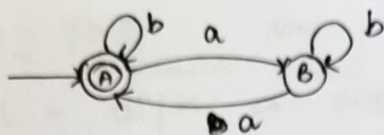
Put (5) in (1),

$$q_1 = q_1 a + q_1 b (b + ab)^* a a + \epsilon$$

$$q_1 = \epsilon + q_1 [a + b(b + ab)^* aa]$$

$$\boxed{q_1 = (a + b(b + ab)^* aa)^*}$$

Ques:



$$A = Ab + Ba + E \quad - (1)$$

$$\frac{B}{R} = \frac{Aa}{Q} + \frac{Bb}{RP} \quad - (2)$$

$$B = Aab^* \quad - (3)$$

Put (3) in (1),

$$A = Ab + Aab^*a + E$$

$$A = E + A(b + ab^*a)$$

$$A = (b + ab^*a)^*$$

# Pumping lemma:-

→ For any regular language  $L$ , there exists an integer  $P$ , such that for all  $w$  in  $L$   $|w| \geq P$ , we can break  $w$  into three strings,  $w = xyz$  such that,

$$1) |xy| < P$$

$$2) |y| > 1$$

3)  $\forall k \geq 0$ : the string  $xy^kz$  is also in  $L$ .

Ques:  $L = \text{even no of 0's \& even 1's}$ . Proof that  $L$  is regular.

1) DFA

2) Regex

Ques:  $L = \{0^n 1^n \mid n \geq 0\}$  Proof that  $L$  isn't regular.

$$w = \underbrace{000}_x \underbrace{111}_y \in L$$

let

$$K = 2$$

then

$$xy^2z = xyxz$$

$$= 00010111 \notin L$$

so this  $L$  isn't regular.

Ques.

$$L = \{ww^R \mid |w| = 2, \Sigma = \{a, b\}\}$$

→ Any language which is finite is always regular.

Ques.

$$L = \{ww^R \mid \Sigma = \{a, b\} \text{ \& } |w| \geq 0\}$$

$$w = aabbbbbaa$$

$$x = a, y = a, z = a$$

$$aabbbbbaa \notin L$$

Ques.

$$L = \{ww \mid w \in (a, b)^*\}$$

$$w = aabaaab$$

$$x = a, y = a, z = a$$

$$aabaaab \notin L$$

$$aabaabaab \notin L$$

Ques.

$$L = \{a^i b^j \mid i, j \geq 1\}$$

$$\rightarrow aa^*bb(bb)^*$$

Regular.

Q  $\Sigma = \{a\}$   $\text{LW}$

- 1)  $L = a^n | n \text{ is even.} \rightarrow \text{Regular}$   $(aa)^*$
- 2)  $L = a^n | n \text{ is odd.} \rightarrow \text{Regular}$   $a(aa)^*$
- 3)  $L = a^n | n \text{ is prime.} \rightarrow \text{Not Regular}$
- 4)  $L = a^{n^2} | n \geq 1 \rightarrow a^1, a^4, a^9, a^{16} \dots$  Not Regular.
- 5)  $L = a^{2^n} | n \geq 1 \rightarrow \text{Not Regular}$

\* If follows AP, may be regular.

\* If follows GP or HP, surely not regular.

H/W

1.  $a^i b^{2^n} | i, n \geq 1$
2.  $w | n_a(w) = n_b(w)$
3.  $a^n b^n c^n | n \geq 1$
4.  $a^n b^n c^m d^m | n, m \geq 1$
5.  $wxw^R | w, x \in (0,1)^+$  Regular
6.  $wxw^R | w \in (0,1)^+ \text{ \& } x \text{ is a special symbol.}$
7.  $wxw^R | x, w \in (0,1)^+, |x| = 5$  NR.
8.  $xww^Ry | x, y, w \in (0,1)^+$  R.
9.  $xww^R | x, w \in (0,1)^+$
10.  $ww^Ry | w, y \in (0,1)^+$